

PrepME AI — Day 2 Learning Notes

1. Project Revision and Testing Workflow

A full-stack project usually has:

```
Frontend (client)
Backend (server)
```

Both run separately and must be started independently.

2. Frontend Server

Frontend built using:

- React
- Vite

To start frontend:

```
cd client
npm run dev
```

This starts React development server.

Usually runs on:

```
localhost:5173
```

Purpose:

- render UI
 - run React app in browser
-

3. Backend Server

Backend built using:

- Node.js
- Express.js

To start backend:

```
cd server  
node index.js
```

Usually runs on:

```
localhost:5000
```

Purpose:

- handle APIs
- process requests
- connect database

4. Full Stack Request-Response Flow

Current application flow:

```
User enters form  
↓  
React state stores values  
↓  
Axios sends POST request  
↓  
Backend API receives request  
↓  
Express parses JSON  
↓  
req.body contains data  
↓  
Backend sends JSON response
```

↓
Frontend updates UI

This is the foundation of full-stack development.

5. Database Introduction

Before database:

- data disappears after refresh/restart

After database:

- data stored permanently

Purpose of database:

- store users
 - passwords
 - notes
 - progress
 - application data
-

6. MongoDB

MongoDB is a NoSQL database.

Why MongoDB?

- beginner friendly
 - flexible
 - widely used in MERN stack
 - stores JSON-like data
-

7. MongoDB Documents and Collections

MongoDB stores:

Documents

Example:

```
{
  "name": "Shweta",
  "email": "abc@gmail.com"
}
```

Collections

Group of documents.

Example:

```
users collection
```

8. MongoDB Atlas

MongoDB Atlas = cloud-hosted MongoDB service.

Benefits:

- free cluster available
- accessible online
- no local DB installation required

Important setup steps:

- create cluster
 - create database user
 - allow network access
 - get connection string
-

9. Connection String

MongoDB connection string looks like:

```
mongodb+srv://username:password@cluster.mongodb.net/
```

Purpose:

- connect backend to cloud database
-

10. Mongoose

Installed using:

```
npm install mongoose
```

Mongoose is a library that helps Node.js work with MongoDB.

Purpose:

- connect database
- create schemas/models
- query database
- manage collections

Think of Mongoose as:

```
Bridge between Node.js and MongoDB
```

11. dotenv

Installed using:

```
npm install dotenv
```

Purpose:

- store secret values safely

Examples:

- DB passwords

- API keys
 - secret tokens
-

12. .env File

Created inside:

```
server/.env
```

Example:

```
MONGO_URI=your_connection_string
```

Important:

- never push secrets publicly
 - .env usually ignored using .gitignore
-

13. process.env

Environment variables accessed using:

```
process.env.MONGO_URI
```

Purpose:

- safely read values from .env file
-

14. Database Connection Code

Example:

```
mongoose.connect(process.env.MONGO_URI)
  .then(() => {
    console.log("MongoDB connected")
  })
```

```
.catch((error) => {  
  console.log(error)  
})
```

15. Promise Handling

.then()

Runs when operation succeeds.

Example:

```
.then(() => {  
  console.log("Connected")  
})
```

.catch()

Runs when error occurs.

Example:

```
.catch((error) => {  
  console.log(error)  
})
```

16. Debugging Connection Errors

Important real-world lesson:

A lot of backend debugging involves:

- wrong credentials
- wrong connection strings
- config mistakes
- port issues
- environment variables

These are extremely common even for experienced developers.

17. URL Encoding

Special characters in passwords can break URLs.

Examples:

Character	Encoded
@	%40
#	%23
&	%26
/	%2F
:	%3A

Example

Real password:

abc@123

Connection string version:

abc%40123

Important:

- actual password does NOT change
 - only URL representation changes
-

18. Backend Restart Workflow

Whenever backend code changes:

Stop server:

```
CTRL + C
```

Restart:

```
node index.js
```

This is required because Node server does not auto-refresh by default.

19. Current Architecture Understanding

Current application structure:

```
Frontend (React)
  ↓
Backend (Express)
  ↓
Database (MongoDB)
```

20. Important Concepts Learned Till Now

Frontend

- React
- components
- props
- state
- forms
- controlled inputs
- useEffect
- Axios

Backend

- Express server
- routes

- APIs
 - GET/POST requests
 - req/res
 - middleware
 - express.json()
 - CORS
-

Database

- MongoDB
 - Atlas
 - Mongoose
 - environment variables
 - DB connection
-

Tools

- Git
 - GitHub
 - npm
 - package management
-

21. Most Important Understanding So Far

The goal is NOT memorizing syntax.

The goal is understanding:

- how data flows
 - how frontend communicates with backend
 - how backend communicates with database
 - how modern applications are structured
-

22. Current Project Status

Completed:

- frontend setup
- backend setup
- frontend-backend communication

- API creation
 - login form flow
 - POST requests
 - MongoDB connection
 - environment variable setup
 - Git/GitHub workflow
-

23. Next Learning Goal

Next phase:

User Model + Database Storage

Will learn:

- schemas
- models
- collections
- saving users
- CRUD basics
- persistent application data